LOW LEVEL DESIGN

The LLD Runbook

A working engineer's guide to the design interview.

LLD is the round where months of preparation can quietly go in the wrong direction. You can know every design pattern by name and still lose the room, because the round isn't testing recall. It's testing whether your design survives the next question.

This is the shortest honest path I know from *I understand OOP* to *I can defend a design under pressure*. Part 1 builds the vocabulary. Part 2 puts it to work on real problems.

Varun Kashyap

Contents

How to use this runbook

This book is short on purpose. Not because low level design is a small subject, but because the version of it that matters in an interview is much smaller than the literature around it.

How it's built

Part 1 is theory, and it is deliberately compressed — principles, patterns and a repeatable framework, with just enough code to make each idea concrete. Part 2 is where the real work happens: ten problems worked end to end, two pages each, in the same order you'd actually solve them in a room.

Read Part 1 once, cover to cover. Come back to it only through Part 2, when a problem makes you reach for something and you want to check that you're reaching for the right thing. Theory read on its own tends not to stick; theory read because a problem demanded it usually does.

Every numbered section is exactly one page, so you can always see how much of a topic is left before it ends.

I ASSUME YOU ALREADY KNOW

- A language with classes and interfaces — Java throughout, but nothing here is Java-specific.
- Basic data structures, enough to reason about a map or a linked list without looking them up.
- What an interface is, even if you've never thought hard about when to introduce one.

I DELIBERATELY SKIP

- Language syntax and OOP mechanics you can look up in a minute.
- The full pattern catalogue. Six patterns carry most interviews; the rest get a line each.
- Distributed systems concerns. That's a different round, and mixing the two costs candidates marks.

Three ways to read this

YOU HAVE	READ THIS
A few weeks	Part 1 cover to cover, then one problem from Part 2 each day. Attempt the design yourself before reading the worked answer.
A weekend	Sections 1.3, all of chapter 3, then 5.1 and 6.1. Follow with the LRU cache and HashMap problems.
Tonight	Sections 1.3, 5.1 and 6.1, then the pattern cheat sheet at the back of Part 2. That is the irreducible core.

A note on the code

Snippets here are short on purpose — usually the ten or so lines where the decision actually lives, with the scaffolding stripped out. Nobody learns anything from a full class listing printed on a page. Where a problem has a complete implementation worth reading, you'll find a link beside it:

`github.com/varunkashyap-vkd/lld-runbook`

All of it compiles and runs. If you learn by breaking things, clone it first and read the chapter second.

1.1 What LLD actually is

Ask ten engineers to define low level design and you'll get ten answers, most of them gesturing vaguely at classes and UML. That vagueness is the first thing working against you, so it's worth fixing before anything else.

A definition worth keeping

Low level design is turning a loosely worded requirement into classes, responsibilities and relationships a team could build on — and then defending those choices when the requirement changes.

Notice the two halves. The first is modelling. The second is defending. Almost everyone prepares for the first, and then loses the round in the second, because the interviewer's real question was never *can you draw this*. It was *what happens to your design when I change my mind*.

Where it shows up

- **A dedicated LLD round.** Forty-five to sixty minutes, a deliberately open prompt — design a parking lot, a rate limiter, a food delivery app. Code is usually optional; structure and reasoning are not.
- **A machine coding round.** Same modelling problem, but you ship running code in ninety minutes to three hours. The design bar is lower, the execution bar is much higher.
- **The last ten minutes of a DSA round.** Your solution works, and then the interviewer says *now make the eviction policy configurable*. The round just turned into LLD without anyone announcing it.
- **A hiring manager or bar-raiser conversation.** No code at all. You talk through a design you've built, and every trade-off you glossed over gets probed.

> WORTH KNOWING
>
> The third one catches the most people out. Candidates relax once the algorithm passes, and treat the follow-up as a bonus question rather than the part that's actually being scored. It's usually the part that's actually being scored.

What changes with your level

LEVEL	WHAT THEY NEED TO SEE	WHAT SINKS YOU
Entry	A correct, readable class structure. Sensible names, responsibilities in roughly the right place.	Procedural code wearing a class costume — one manager object doing everything.
Mid	Justified trade-offs. You chose composition here and an interface there, and can say why out loud.	Design on autopilot. Patterns applied because they're familiar, not because the problem asked.
Senior	Questioning the requirement itself, and drawing boundaries that hold when the follow-up lands.	Answering exactly what was asked, and nothing more.

1.2 Four rounds that get confused

DSA, LLD, machine coding and HLD blur together in most preparation material, and the blur is expensive. Knowing which room you're in changes what a good answer looks like.

	DSA	LLD	MACHINE CODING	HLD
Unit of work	An algorithm	A class model	A running module	Services and data flow
You optimise for	Correctness, then complexity	Structure that absorbs change	Working software, quickly	Behaviour at scale
Really scored on	Does it work, and how fast	Why these classes, and not others	Does it run, and is it clean	Trade-offs across the system
Typical length	45 min	45–60 min	90–180 min	45–60 min
Classic failure	Brute force, never optimised	Writing code before naming the nouns	Over-engineering until nothing runs	Naming technologies instead of reasoning

The gear shift that trips people up

DSA trains a specific instinct: there is one best answer, and your job is to find it. That instinct actively hurts you in an LLD round, where there is a family of defensible answers and the interviewer is watching how you choose between them. Two candidates can produce different class diagrams for the same prompt and both clear the bar — the one who can't explain their choice is the one who doesn't.

The practical consequence is about silence. In a DSA round, going quiet for two minutes while you think is normal, sometimes even a good sign. In an LLD round it's close to fatal, because the reasoning *is* the deliverable. A diagram that appears on the board without narration gives the interviewer nothing to score.

> **THE LINE BETWEEN LLD AND HLD**
>
> If your answer involves load balancers, replication or a message queue, you've drifted into HLD. If it involves which class owns a field and who is allowed to mutate it, you're in LLD. Drifting upward feels impressive and reads as avoidance — it's the most common way experienced candidates lose points on a problem they could have modelled well.

When you can't tell which room you're in

Ask. *Would you like me to focus on the class design, or do you want something running by the end?* is a perfectly reasonable opening question, and I've never seen it land badly. Recruiters describe rounds inconsistently, interviewers adapt on the day, and the thirty seconds you spend clarifying is far cheaper than twenty minutes spent solving the wrong problem well.

1.3 What's actually being evaluated

Interviewers rarely score the diagram. They score the sequence of decisions that produced it, and there are five of them worth preparing for directly.

The five things on the scorecard

1. **Requirement clarification.** Did you find the ambiguity before you built on top of it? Prompts are underspecified on purpose.
2. **Entity modelling.** Are your nouns the right nouns? Most bad designs are traceable to a class that should never have existed, or one that should have.
3. **Boundaries and abstraction.** What is hidden behind what, and can a caller do damage you didn't intend?
4. **Extensibility under pressure.** The follow-up change — how much of your design has to be rewritten to absorb it?
5. **Communication.** Whether the four above were visible to the person in the room, or happened silently in your head.

How strong DSA candidates lose this round

- **Coding too early.** Ten minutes of unprompted code buys you a structure you now feel obliged to defend.
- **Pattern-dropping.** A Factory, a Singleton and an Observer on a problem that needed one interface. It reads as insecurity, not fluency.
- **Designing for imaginary scale.** Thread pools and caches for a problem with three entities and no stated load.
- **Going quiet.** Covered on the previous page, and worth repeating — unnarrated thinking is unscored thinking.
- **Treating the follow-up as an attack.** Defensiveness here is a bad look. The change was always coming; it's the whole point of the round.

What good sounds like

The gap between a pass and a strong pass is usually one sentence pattern, repeated all the way through: **state the choice, name the alternative, give the reason.**

> "I'll put eviction behind an interface rather than a flag on the cache. The flag is less code today, but every new policy means editing the same class — and I'd rather add a file than edit one."

That takes twenty seconds and covers three of the five items above. Say it out loud enough times in practice and it stops sounding rehearsed.

> **IF YOU TAKE ONE THING FROM THIS CHAPTER**
>
> The follow-up isn't a curveball, and it isn't the interviewer being difficult. It's the exam. Everything before it exists to set up the moment where the requirement shifts and we find out whether your design bends or breaks. Chapter 6 turns that into a repeatable sequence you can run on any prompt.

2.1 Encapsulation

Encapsulation is not "make fields private and add getters". It is deciding who is allowed to change state, and then refusing everyone else.

The useful question is never *what is the visibility of this field*. It is *what does this class promise about itself, and could a caller break that promise?* A cart whose total can disagree with its items is broken no matter how private the fields are.

ANTI-PATTERN

```
class Cart {
    public List<Item> items = new ArrayList<>();
    public Money total;                 // who keeps this in sync?
}
```

BETTER

```
class Cart {
    private final List<Item> items = new ArrayList<>();

    void add(Item item) { items.add(item); }

    List<Item> items() { return List.copyOf(items); }   // no outside mutation

    Money total() {                                     // derived, never stale
        return items.stream().map(Item::price)
                    .reduce(Money.ZERO, Money::add);
    }
}
```

Two decisions did the work. The total is computed rather than stored, so there is no second copy of the truth to fall out of step. And the list is handed out as a copy, so a caller holding a reference cannot quietly add to it.

Common mistakes

- **Getter and setter for every field.** A private field with a public setter is a public field with extra typing. Ask what the setter protects.
- **Returning the internal collection.** The most common one. It hands every caller a key to your state.
- **Validating in the caller.** If the object can be constructed in an invalid state, someone eventually will.

WHAT THE INTERVIEWER IS CHECKING

Whether you can name the invariant out loud. "A cart's total must always equal the sum of its items" is the sentence that justifies every access decision on the page. Say it and the design explains itself; skip it and you are just guessing at keywords.

2.2 Abstraction

Encapsulation hides state. Abstraction hides decisions — it stops callers from knowing *how* something is done, so you can change your mind later.

ANTI-PATTERN

```
class OrderService {
    void place(Order order) {
        SmtpClient smtp = new SmtpClient("smtp.internal", 587);
        smtp.send(order.customerEmail(), "Order placed");
    }
}
```

Nothing here is wrong until the day you add SMS. Then `OrderService` — a class about orders — has to be edited because of a messaging decision. It knows a hostname and a port number, and neither has anything to do with placing an order.

BETTER

```
interface Notifier {
    void notify(Customer to, String message);
}

class OrderService {
    private final Notifier notifier;

    OrderService(Notifier notifier) { this.notifier = notifier; }

    void place(Order order) {
        notifier.notify(order.customer(), "Order placed");
    }
}
```

How a leaking abstraction announces itself

Watch the names. The moment a technology appears in the contract — `sendEmail()`, `getConnection()`, `toJson()` — the abstraction has failed, because every caller now depends on the choice you were trying to hide.

Interface or abstract class

	USE IT WHEN	COST
Interface	You need a contract and nothing more. This is the default.	No shared code; each implementation stands alone.
Abstract class	Subclasses genuinely share state or a fixed algorithm skeleton.	Spends the one inheritance slot and couples subclasses to the base.

Common mistakes

- **An interface with one implementation, added "for flexibility".** If no second implementation is coming, that is indirection, not abstraction.
- **Methods only one implementation can honour.** The rest throw, and the contract was a lie.

2.3 Inheritance

Inheritance is the most over-used relationship in LLD interviews. It buys you code reuse and charges you coupling, and candidates rarely price the second half.

ANTI-PATTERN

```
class Stack extends ArrayList<String> {
    void push(String s) { add(s); }
    String pop() { return remove(size() - 1); }
}
```

This compiles, passes a quick test, and is broken. `Stack` did not inherit two useful methods — it inherited the *entire* `ArrayList` API. A caller can now write `stack.add(0, "x")` and insert at the bottom, or call `clear()`, or index into the middle. The one thing a stack promises is the one thing this class cannot enforce.

BETTER

```
class Stack {
    private final List<String> items = new ArrayList<>();

    void push(String s) { items.add(s); }

    String pop() {
        if (items.isEmpty()) throw new NoSuchElementException();
        return items.remove(items.size() - 1);
    }
}
```

Same reuse, none of the exposure. The list is now a private implementation detail rather than a public inheritance contract.

When inheritance is genuinely right

All three have to hold, not just the first:

- The relationship is a real *is-a*, and callers will substitute the subtype for the base.
- The base was designed for extension — stable, documented, and yours to change.
- The subclass weakens no guarantee the base makes.

Common mistakes

- **Inheriting for reuse rather than substitutability.** If you never treat the subtype as the base, you wanted a field, not a parent.
- **Overriding a method to throw.** `UnsupportedOperationException` is an admission that the hierarchy is wrong.

WHAT THE INTERVIEWER IS CHECKING

Whether `extends` is your reflex. Reaching for composition first, and giving a reason when you do inherit, separates people who learned OOP from people who have maintained it. If you inherit, say which guarantee the base makes and why your subclass keeps it.

2.4 Polymorphism

Textbooks explain polymorphism with animals making sounds. In an interview its value is far more concrete: it is how you delete a conditional that would otherwise grow forever.

ANTI-PATTERN

```
Money fee(Payment p) {
    switch (p.type()) {
        case CARD:   return p.amount().percent(2);
        case UPI:    return Money.ZERO;
        case WALLET: return p.amount().percent(1);
    }
    throw new IllegalArgumentException(p.type().name());
}
```

The problem is not this method. It is that a codebase like this never has only one — there is a matching switch for settlement time, another for refund rules, another for the receipt label. Adding a payment type means finding all of them, and the compiler will not tell you when you miss one.

BETTER

```
interface PaymentMethod {
    Money fee(Money amount);
}

class Card implements PaymentMethod {
    public Money fee(Money amount) { return amount.percent(2); }
}

class Upi implements PaymentMethod {
    public Money fee(Money amount) { return Money.ZERO; }
}
```

Adding a wallet is now a new file rather than an edit to four existing ones. That is the Open/Closed Principle arriving early — we come back to it in **3.2**, and it is the backbone of the Strategy pattern in **5.2**.

When a switch is the right answer

On a closed set that genuinely will not grow — days of the week, HTTP methods — a switch is clearer than four classes. Replacing every conditional with a type hierarchy is its own failure mode, and an experienced interviewer will read it as over-engineering.

Common mistakes

- `instanceof` **chains.** The same growing conditional wearing a disguise. If you are asking what type something is, the type should be deciding.
- **Logic living in the caller.** Data-only classes with all the behaviour outside them means polymorphism has nowhere to happen.

3.1 Single Responsibility

Usually taught as "a class should do one thing", which is useless in practice — everyone believes their class does one thing.

What it actually says

The sharper phrasing is that a class should have **one reason to change**. Not one method, not one job in some vague sense: one source of future change requests. Two things belong in the same class when they will always change together, and belong apart when they change for different reasons at different times.

> The test I actually use: *who files the bug report?* If finance asks for a tax change and marketing asks for a wording change, and both edits land in the same file, that file has two responsibilities.

ANTI-PATTERN

```
class Invoice {
    Money subtotal()          { ... }
    Money tax()               { ... }   // finance changes this
    byte[] toPdf()            { ... }   // design changes this
    void email(Customer c)    { ... }   // marketing changes this
    void save()               { ... }   // infrastructure changes this
}
```

Four departments, one file. Every release, four different people touch it for unrelated reasons, and each of them risks breaking the other three.

BETTER

```
class Invoice {                          // what an invoice IS
    Money subtotal()               { ... }
    Money total(TaxPolicy policy)  { ... }
}

class InvoicePdfRenderer { byte[] render(Invoice invoice)      { ... } }
class InvoiceMailer      { void send(Invoice invoice, Customer to) { ... } }
class InvoiceRepository  { void save(Invoice invoice)          { ... } }
```

Common mistakes

- **Splitting by line count.** A long class is not automatically a violation, and a short one is not automatically fine.
- **Over-splitting.** Eight classes to follow one flow is its own failure. SRP is about axes of change, not about making files small.

WHAT THE INTERVIEWER IS CHECKING

Not whether you can split a class — anyone can split a class. Whether you can say why the lines are where they are. "These two always change together, that one changes for a different reason" is the sentence that turns an arbitrary split into a design.

3.2 Open/Closed

"Open for extension, closed for modification" is the most quoted and least understood line in SOLID. Read literally it says never edit your code, which is absurd.

What it actually says

Pick the axis along which you know the system will vary, and put a seam there. New variations of that *known kind* should arrive as new code, while the code that already works and is already tested stays untouched.

The honest part that gets left out: you cannot be open to everything at once. Choosing which axis to protect *is* the design decision. Guess right and change is cheap forever. Guess wrong and you have paid for indirection that never pays you back.

> Ask yourself: *what will the next ten feature requests have in common?* If the answer is "another payment type" or "another discount", that is your axis.

ANTI-PATTERN

```
class Checkout {
    Money discount(Cart cart, String code) {
        if (code.equals("FLAT10"))   return Money.of(10);
        if (code.equals("PCT20"))    return cart.total().percent(20);
        if (code.equals("FREESHIP")) return cart.shipping();
        return Money.ZERO;
    }
}
```

Every campaign the marketing team dreams up means editing — and re-testing — a class that is on the critical path of taking money.

BETTER

```
interface Discount {
    Money apply(Cart cart);
}

class PercentOff implements Discount {
    private final int percent;
    public Money apply(Cart cart) { return cart.total().percent(percent); }
}

class Checkout {
    Money discount(Cart cart, Discount discount) { return discount.apply(cart); }
}
```

This is the same move as **2.4**, and it is the skeleton of the Strategy pattern in **5.2**. Three ideas, one shape.

Common mistakes

- **A seam for an axis that never varies.** Speculative flexibility is the most expensive kind of wrong guess: you pay the indirection cost every time someone reads the code, and collect nothing.
- **Reading it as "never edit existing code".** Bug fixes modify code. The principle is about absorbing *new variations* without reopening what works.

3.3 Liskov Substitution

The formal statement scares people off. The practical version is short: if code works with a base type, it must keep working when handed any subtype — without being told which one it got.

What it actually says

Inheritance is a promise made to *callers*, not a convenience for implementers. When you extend a type you inherit its guarantees, and you are not allowed to quietly take any of them away.

> A subtype may accept more than the base and guarantee more than the base. It may never accept less or guarantee less. **Do more, never less.**

ANTI-PATTERN

```
class Document {
    void save() { writeToDisk(); }
}

class ReadOnlyDocument extends Document {
    @Override void save() {
        throw new UnsupportedOperationException();
    }
}
```

Now every loop of the form `for (Document d : docs) d.save();` is a runtime bomb, and the caller has no way to know before it goes off. The subtype took away a guarantee the base had made.

BETTER

```
interface Document {
    String content();
}

interface WritableDocument extends Document {
    void save();
}
```

The distinction moves into the type system. Code that needs to save asks for a `WritableDocument`, and a read-only document simply cannot be passed to it.

HOW VIOLATIONS SHOW UP IN REVIEW

Look for callers that must know the concrete type — an `instanceof` check before a polymorphic call, or a `try/catch` wrapped around one "just in case". Both mean substitution has already failed, whatever the class diagram claims.

Common mistakes

- **A subtype that rejects input the base accepted.** If the base allows a null discount and your subclass throws on it, callers written against the base are now wrong through no fault of their own.
- **Throwing something new.** An exception the base never declared is a broken promise, even when the signature still compiles.

3.4 Interface Segregation

No class should be forced to depend on methods it does not use. It is the smallest of the five principles and the easiest to check — you can usually spot a violation by scrolling.

What it actually says

A wide interface hurts twice. Implementers are forced to supply methods that make no sense for them, and callers end up depending on capabilities they never invoke — so a change to a method you don't use can still force you to recompile, retest, and occasionally rewrite.

> The giveaway is an `@Override` with an empty body or a thrown exception. Every one of those is the type system telling you the interface is too wide.

ANTI-PATTERN

```
interface Job {
    void start();
    void pause();
    void resume();
    void cancel();
}

class ReportJob implements Job {
    public void start()  { ... }
    public void pause()  { }        // nothing sensible to do
    public void resume() { }
    public void cancel() { throw new UnsupportedOperationException(); }
}
```

BETTER

```
interface Job         { void start(); }
interface Pausable    { void pause(); void resume(); }
interface Cancellable { void cancel(); }

class ReportJob implements Job                { ... }
class ExportJob implements Job, Cancellable   { ... }
```

Each job now declares exactly what it can do, and scheduling code that only ever cancels can ask for `Cancellable` and ignore the rest.

WORTH NOTICING

The bad version above is also a Liskov violation — `cancel()` throwing is the exact failure from **3.3**. That is not a coincidence. Fat interfaces are the usual cause of broken substitution, so segregating them tends to fix both at once.

One mistake worth naming

Segregate by **role**, not by implementation. Creating one interface per concrete class — `ReportJobInterface`, `ExportJobInterface` — looks like segregation and achieves nothing. The question is what a *caller* needs, not who happens to implement it.

3.5 Dependency Inversion

Two things get confused here. Dependency inversion is not dependency injection, and it is not "use interfaces everywhere". Injection is a technique; inversion is a direction.

What it actually says

High-level policy — the code expressing what your product does — should not depend on low-level detail such as a database, an HTTP client or a file format. Both should depend on an abstraction instead.

The half that candidates almost always miss is *who owns the abstraction*. The interface belongs to the module that **needs** it, not the one that implements it. That ownership flip is the inversion; without it you have merely added an interface and pointed the same arrow through it.

ANTI-PATTERN

```
class OrderService {
    private final MySqlOrderRepository repo = new MySqlOrderRepository();

    void place(Order order) { repo.insert(order); }
}
```

Order policy now depends on MySQL. You cannot test placing an order without a database, and swapping the store means editing business logic.

BETTER

```
interface OrderRepository {          // declared beside OrderService
    void save(Order order);
}

class OrderService {
    private final OrderRepository repo;

    OrderService(OrderRepository repo) { this.repo = repo; }

    void place(Order order) { repo.save(order); }
}
```

`MySqlOrderRepository` now implements an interface defined by the domain, so the dependency arrow runs from detail toward policy. Assembly happens once, at the edge of the application, where `main` or a configuration class wires the concrete pieces together.

Common mistakes

- `new` **inside a policy class.** Every `new` on a collaborator is a decision you can no longer change from the outside.
- **Putting the interface in the infrastructure package.** If the detail owns the abstraction, nothing was inverted — you added a file and kept the arrow.

WHAT THE INTERVIEWER IS CHECKING

The fastest signal is testability. "How would you unit test this?" is really asking whether your dependencies can be replaced. If the answer needs a running database, the arrow is pointing the wrong way.

4.1 KISS, DRY and YAGNI

Chapter 3 was about adding structure. This one is the counterweight — three habits that tell you when to stop.

These are rarely asked by name. No interviewer opens with "what is YAGNI?" They are scored silently instead: someone notices that you did not build a plugin architecture for a parking lot, and that observation quietly becomes "pragmatic, would ship".

PRINCIPLE	WHAT IT SAYS	OVER-APPLIED, IT BECOMES
KISS Keep It Simple, Stupid	Choose the simplest design that fully solves the problem actually in front of you.	Refusing structure the problem genuinely needs, until everything lives in one method.
DRY Don't Repeat Yourself	Every piece of knowledge should have one authoritative home, so a change lands in exactly one place.	Merging code that merely looks alike, welding together things that needed to evolve apart.
YAGNI You Aren't Gonna Need It	Don't build it until a real requirement asks for it.	Ignoring a change you already know is coming, and paying to retrofit it.

The one that does real damage

DRY causes more bad designs than the other two combined, because it is misremembered as "no two pieces of code may look the same".

LOOKS LIKE DUPLICATION

```
class SignupValidator    { boolean valid(String email) { return email.contains("@"); } }
class NewsletterValidator{ boolean valid(String email) { return email.contains("@"); } }
```

Extract a shared base class and you have just coupled signup policy to newsletter policy. Two months later signup must reject free email domains — and now you are unpicking an abstraction that never should have existed. The code was identical by coincidence, not because it expressed the same rule.

> DRY is about duplicated **knowledge**, not duplicated **text**. The test: if this rule changes, must both places change? If they would change independently, leave them alone.

How this shows up in the room

- A configuration-driven rules engine for a problem that has three rules.
- A base class extracted because two methods happened to look similar.
- A caching layer, a thread pool and a retry policy nobody asked for.

WHEN THESE FIGHT CHAPTER 3

Open/Closed says put a seam where the system will vary. YAGNI says do not build for requirements nobody has stated. Both are right, and on any given decision they point in opposite directions.

The resolution is to say it out loud: add the seam when you can *name* the second case, and write the simple version when you cannot. "I'd keep this a switch until we have a second payment type, then it becomes a Strategy" is a stronger answer than either principle recited on its own.

4.2 Composition over inheritance

Section 2.3 argued that inheritance is over-used and showed how it leaks. This is the positive case — what to reach for instead, and why it keeps working as requirements pile up.

The argument is arithmetic

A class hierarchy can only encode one axis of variation cleanly. The moment a second axis appears, you are not adding classes — you are multiplying them.

TWO AXES, ONE HIERARCHY

```
class CsvEmailReport extends Report { ... }
class CsvS3Report    extends Report { ... }
class PdfEmailReport extends Report { ... }
class PdfS3Report    extends Report { ... }   // add JSON: six. add FTP: nine.
```

Format and destination are independent choices, but the hierarchy forces every combination to exist as its own class. Each new format multiplies by the number of destinations, and the shared code inside them starts getting copied.

TWO AXES, TWO ABSTRACTIONS

```
class Report {
    private final Formatter formatter;
    private final Destination destination;

    Report(Formatter formatter, Destination destination) {
        this.formatter = formatter;
        this.destination = destination;
    }

    void publish(Data data) {
        destination.send(formatter.format(data));
    }
}
```

> With inheritance you **multiply**. With composition you **add**. Adding JSON is one new class, and it works with every destination that already exists.

What it costs

Composition is not free. Somebody has to assemble the pieces, so object creation gets wordier and one more indirection sits between the caller and the work. That price is worth paying when the axes are genuinely independent, and not worth paying for a single fixed variation — which is exactly the judgement **4.1** is about.

This shape returns twice: as the Strategy pattern in **5.2**, and as Decorator in **5.6**.

When inheritance still wins

"Prefer composition" is a default, not a prohibition. Inheritance earns its place when the subtype is a genuine specialisation that callers will pass anywhere the base is expected, and when the base is stable, documented and yours to change.

Template Method — a base class that fixes the steps of an algorithm and lets subclasses fill in individual ones — is the one common pattern that genuinely depends on inheritance. It appears in **5.7**.

4.3 Interfaces and immutability

Two defaults worth adopting. Neither is dramatic on its own, and together they remove a whole category of bug from a design.

Program to interfaces

This is about what you *depend on*, not about creating more interfaces — that distinction matters, because **2.2** warned against inventing an interface per class. Here the rule is narrower: declare things by the weakest type that does the job.

DEPEND ON THE CONTRACT, NOT THE IMPLEMENTATION

```
List<Order> orders = new ArrayList<>();      // not ArrayList<Order> orders

void audit(Collection<Order> orders) { ... }   // I only need to iterate
```

Widening the parameter to `Collection` costs nothing and immediately makes the method usable with a set. Narrowing it to `ArrayList` gains nothing and rules out every other caller.

Favour immutability

An object that cannot change cannot be corrupted, cannot surprise a caller who stored a reference to it, is thread-safe with no effort, and is safe to use as a map key. Very few design decisions pay that well for so little.

A VALUE OBJECT

```
final class Money {
    private final long amount;
    private final Currency currency;

    Money add(Money other) {                // returns a new value
        return new Money(amount + other.amount, currency);
    }
}
```

`Money`, `DateRange`, `Coordinates`, `Address` — anything you would compare by value rather than identity is a candidate. Make them immutable and equality, hashing and sharing all stop being questions you have to answer.

CHEAP TO SAY, HARD TO FAKE

"I'll make `Money` immutable so arithmetic returns new values and equality is safe" takes five seconds and signals more experience than most of what candidates rehearse. It is the kind of remark that only comes from having been bitten.

Common mistakes

- **Calling a class immutable because its fields are `final`.** A `final` reference to a mutable list is still mutable. The field cannot be reassigned; the contents can be changed by anyone holding the same list.
- **Making everything immutable.** Value objects want it. Entities with a real lifecycle — an order that moves through states — do not, and forcing it there produces copies nobody wanted to write.

5.1 Choosing a pattern

Patterns are answers. Almost everyone learns them in that order — answer first, question later — which is why so many candidates can define six patterns and still apply none of them correctly.

So this chapter is organised around the questions. Each pattern that follows opens with the symptom that should make you reach for it, because an interview never asks *what is Observer*. It describes a situation and waits to see whether you notice.

REACH FOR	WHEN YOU SEE THIS
Singleton 5.2	Exactly one instance must exist, and everything needs to reach it.
Factory 5.3	Callers switching on a type code to decide which class to instantiate.
Builder 5.4	Constructors with six parameters, half of them optional and three the same type.
Observer 5.5	One change that several unrelated parts of the system need to hear about.
Strategy 5.6	A conditional that grows a branch every time a new kind of thing appears.
State 5.7	Behaviour that depends on a status field, with transition rules scattered everywhere.
Decorator 5.8	Optional behaviour that applies to some instances, in combinations you cannot enumerate.

Three groups, three questions

- **Creational** — who decides what gets made, and how. Singleton, Factory, Builder.
- **Behavioural** — how objects share work and talk to each other. Observer, Strategy, State.
- **Structural** — how pieces are assembled into a larger whole. Decorator.

When the answer is no pattern at all

Most LLD problems need one pattern, and plenty need none. A parking lot modelled with clear entities, sensible responsibilities and one interface where the pricing rule lives is a complete, passing answer. Reaching for a second and third pattern to prove you know them is the single fastest way to look junior — it reads as decoration, and decoration invites exactly the questions you cannot answer.

> Applying the structure scores; announcing the vocabulary does not. "I'll put eviction behind an interface so the policy can be swapped" earns everything that saying "I'll use Strategy" earns, and it survives the follow-up question. Reaching for a pattern you cannot justify is worse than using none at all.

5.2 Singleton

The most famous pattern in the catalogue, and the one most likely to be used against you. Worth knowing precisely, and worth reaching for carefully.

The signal

One instance must exist, and creating a second would be wrong rather than merely wasteful — a connection pool, an in-memory cache, a configuration registry, an ID generator. Note the emphasis: *wrong*. "It would be a bit expensive to make two" is not a reason to reach for this.

THE VERSION EVERYONE WRITES FIRST

```
class Config {
    private static Config instance;

    static Config getInstance() {
        if (instance == null) instance = new Config();    // two threads, two configs
        return instance;
    }
}
```

LAZY AND THREAD-SAFE, WITHOUT LOCKING

```
class Config {
    private Config() { }

    private static class Holder {
        static final Config INSTANCE = new Config();     // JVM guarantees this once
    }

    static Config get() { return Holder.INSTANCE; }
}
```

The holder class is not loaded until `get()` is first called, and class initialisation is already thread-safe, so you get laziness and safety without a `synchronized` block. If you would rather not explain that, a single-element `enum` is the shortest correct Singleton in Java.

Where it shows up

- Connection and thread pools, where a second pool would quietly exhaust a limit.
- Configuration and feature flags, read from everywhere and written once at startup.
- An in-memory cache — including the LRU cache worked through in Part 2.
- ID and sequence generators, where two instances would hand out the same number.

THE TRAP

A Singleton is global mutable state wearing a design-pattern badge. A class that calls `Config.get()` does not declare that dependency anywhere, so it cannot be tested in isolation and its tests become order-dependent.

The senior answer keeps the benefit and drops the cost: *create one instance, then inject it*. One object, no static reach, and the dependency is visible in the constructor — which is **3.5** doing its job.

5.3 Factory

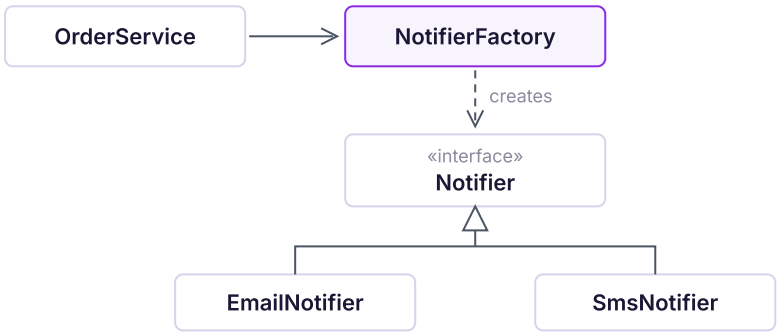
Reach for a factory when the decision of *which class to create* keeps leaking into code that should not care.

The signal

IN THREE SERVICES, WITH THREE SLIGHTLY DIFFERENT VERSIONS

```
if      (channel == EMAIL) notifier = new EmailNotifier(smtpHost, port);
else if (channel == SMS)   notifier = new SmsNotifier(gatewayKey);
else                       notifier = new PushNotifier(fcmToken);
```

Every caller now knows every implementation and every constructor argument. Adding a channel means finding all of them.



Callers depend on `Notifier`; only the factory knows the implementations.

BETTER

```
class NotifierFactory {
    Notifier create(Channel channel) {
        switch (channel) {
            case EMAIL: return new EmailNotifier(smtpHost, port);
            case SMS:   return new SmsNotifier(gatewayKey);
            default:    return new PushNotifier(fcmToken);
        }
    }
}
```

Notice what did **not** happen: the switch did not disappear. Factory does not remove the conditional, it confines it to one place — so adding a channel means editing one file that exists for exactly that purpose.

Shows up in: parking lot vehicle types, notification channels, report and document exporters, shape rendering, payment gateway selection.

WORTH KNOWING

What interviewers usually mean by "factory" is the class above — strictly a *simple factory*. The GoF *Factory Method* is different: the creator itself is subclassed, so each subclass decides its own product. Reach for it only when the creator genuinely varies too; otherwise the simple version is the honest answer.